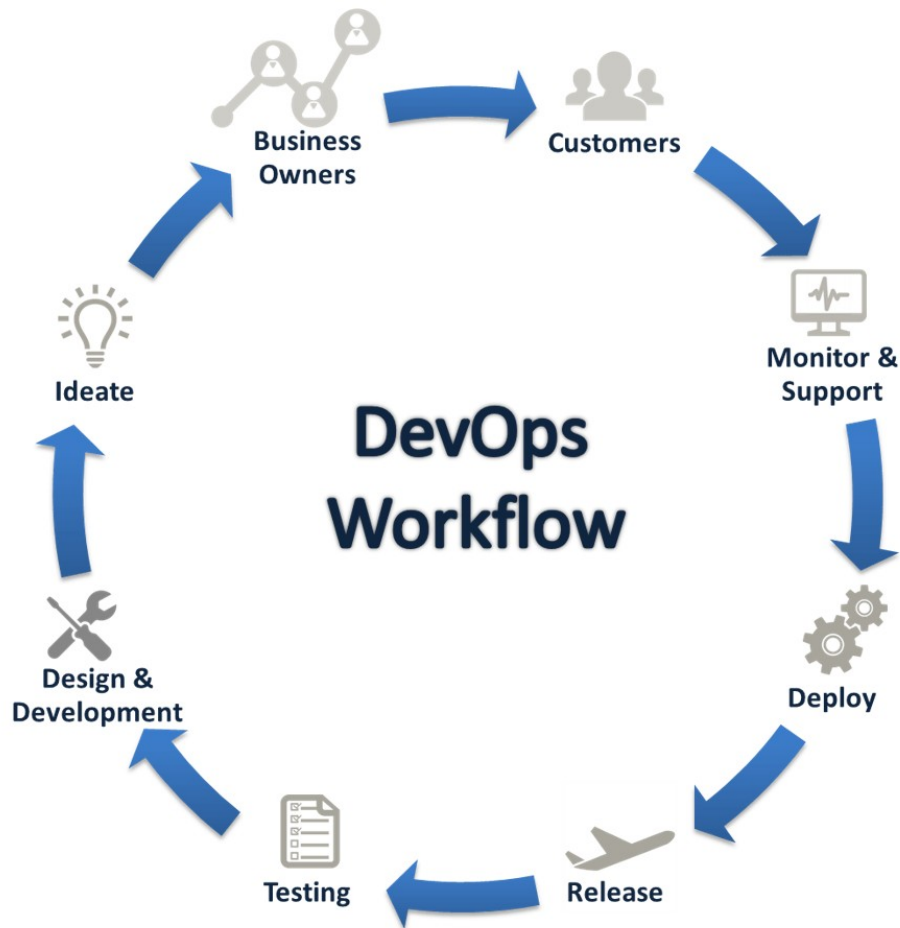


Cursos Especialització: **Desplegament de productes de programari en contenidors**
Família Professional: **Informàtica i comunicacions**
Grau: **Superior**



Mòdul 5167: Definició de fluxos de processos en integració contínua

Índex

📌 Bloc 1: Gestió Avançada de Repositoris de Codi Font i Seguretat Digital

TEMA 1: Arquitectura i Estructuració de Repositoris

TEMA 2: Autenticació, Autorització i Governança en Repositoris

TEMA 3: Validació Automatitzada de Codi (Linterització i Dependències)

TEMA 4: Seguretat del Codi i Gestió de Vulnerabilitats (SAST)

TEMA 5: Manteniment de Repositoris a Gran Escala

📌 Bloc 2: Automatització de Serveis, Configuració d'Artefactes i Fluxos de CI

TEMA 6: Automatització i Creació de Serveis via CLI i API

TEMA 7: Parametrització d'Artefactes en el Cicle de Vida

TEMA 8: Infraestructura com a Codi (IaC) i Contenedors en la IC

TEMA 9: Execució de Pipelines de CI, Diagnòstic i Gestió de Fallades

TEMA 10: Integració de Codi, Resolució de Conflictes i Tancament del Flux

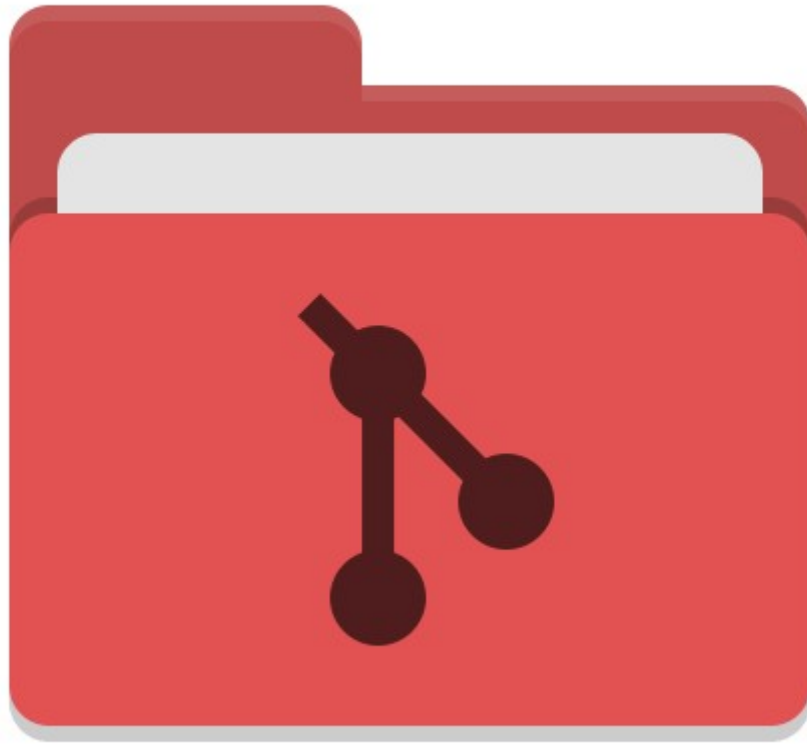
📌 Bloc 3: Eines de Col·laboració i Interconnexió del Flux de Treball

TEMA 11: Plataformes de Comunicació i Gestió de Projectes

TEMA 12: Interconnexió de Sistemes i Automatització de Notificacions

TEMA 13: Traçabilitat d'Errors i Documentació Col·laborativa

TEMA 1: Arquitectura i Estructuració de Repositoris



- 1 Introducció als sistemes de control de versions distribuïts (DVCS)
 2. Estratègia de branques de Git (*Branching Strategies*)
 - 3 Estructuració lògica de branques en CI: Estables vs. canvis en procés
 - 4 Traçabilitat del codi i bones pràctiques en missatges de *Commit*
-  *Pràctica de Laboratori Proposada per a aquest tema*

1 Introducció als sistemes de control de versions distribuïts (DVCS)

Un Sistema de Control de Versions Distribuït (DVCS), com Git, és una eina fonamental en la Integració Contínua (CI). A diferència dels sistemes centralitzats antics (on hi havia un sol servidor central), en un sistema distribuït cada desenvolupador té una còpia completa del repositori en la seua màquina local, incloent-hi l'històric sencer de canvis, branques i etiquetes.

Avantatges clau per a la Integració Contínua:

- **Autonomia:** Permet treballar, fer *commits* i crear branques en local sense connexió a Internet.
- **Velocitat:** Les operacions d'historial i canvis són instantànies perquè es fan en local.
- **Traçabilitat:** Cada modificació queda registrada amb un identificador únic (codi **HASH SHA-1/SHA-256**), un autor, una data i un missatge descriptiu.

2. Estratègia de branques de Git (*Branching Strategies*)

Perquè un equip de desenvolupament treballi de forma coordinada sense xafar-se el codi, és obligatori utilitzar una **estratègia de branques**. Segons el projecte, s'elegeix un model o un altre. Els tres més utilitzats en entorns DevOps són:

A) GitFlow

És un model clàssic, rígid i basat en rols molt definits per a cada branca. És ideal per a projectes amb cicles de llançament tradicionals (versions tancades com v1.0, v2.0).

- **Branques principals i infinites:**
- main (o master): Conté el codi que està actualment en producció. És sagrada i 100% estable.
- develop: Conté el codi de la pròxima versió. És on s'integren les noves funcionalitats.
- **Branques temporals:**
- feature/*: S'obrin des de develop per a desenvolupar una nova característica i es tornen a fusionar a develop.
- release/*: S'obrin des de develop quan es vol preparar una versió per a producció (fase de congelació de codi i correcció d'errors). Es fusiona a main i a develop.
- hotfix/*: S'obrin directament des de main per a solucionar un error crític en producció de forma urgent. Es fusiona a main i a develop.

B) Trunk-Based Development (Desenvolupament basat en el tronc)

És el model preferit en **Integració Contínua Avançada** i organitzacions d'alt rendiment.

- Els desenvolupadors fusionen els seus canvis en una única branca central (anomenada trunk o main) **diverses vegades al dia**.
- S'eviten les branques de llarga durada. Si es creen branques, són molt jueves i duren poques hores.
- **Avantatge:** Redueix dràsticament els conflictes de fusió (*merge hell*).
- **Requisit:** Requereix una suite de tests automatitzats molt potent, ja que qualsevol error pot trencar la branca principal. S'utilitzen tècniques com *Feature Toggles* (activadors/desactivadors de codi en temps d'execució) per a amagar funcionalitats no acabades.

C) GitHub Flow

Un model molt més simple que GitFlow, ideal per a aplicacions web i desplegament continu.

- Només hi ha una branca principal estable (main).
- Qualsevol canvi es fa creant una branca descriptiva (ex: add-login-button).
- Es puja al servidor i s'obri una **Pull Request (PR)** per a debatre els canvis.
- Una vegada validat el codi i passats els tests de CI, es fusiona amb main i es desplega immediatament a producció.

3 Estructuració lògica de branques en CI: Estables vs. canvis en procés

D'acord amb les directives de qualitat i el criteri d'avaluació normatiu, un repositori s'ha d'organitzar separant clarament els entorns d'estabilitat:

1. **Branques de codi estable i validat (main, production):**
 - Representen l'estat del programari que funciona correctament.
 - Estan **protegides** (no es pot fer un git push directe).
 - Només reben codi que ha superat amb èxit totes les validacions del *pipeline* de CI (sintaxi, semàntica, tests i seguretat).
2. **Branques de canvis en procés (develop, feature/*, fix/*):**
 - Són zones de treball dinàmiques on es recullen els canvis diaris.
 - El codi pot ser inestable o estar a mig acabar.
 - L'eina de CI escolta constantment aquestes branques per a executar tests ràpids cada vegada que un programador hi fa un canvi.

4 Traçabilitat del codi i bones pràctiques en missatges de *Commit*

La traçabilitat és la capacitat de rastrejar la història d'un canvi: saber *qui* ho va fer, *quan*, *per què* i *quin tiquet o incidència* ho va motivar.

Per a garantir-la, s'utilitzen estàndards en la redacció de missatges de commit, sent el més reconegut Conventional Commits.

Estructura de Conventional Commits:

```
<tipus>(<àmbit opcional>): <descripció curta en imperatiu>
[cos del missatge opcional]
[peu de pàgina opcional]
```

- **Tipus comuns:**
- feat: Una nova funcionalitat per a l'usuari.
- fix: La resolució d'un error (bug).
- docs: Canvis només en la documentació.

- test: Afegir o corregir proves automatitzades.
- refactor: Canvis en el codi que ni arreglen un bug ni afigen una funcionalitat (millora de disseny).
- chore: Tasques rutinàries o actualitzacions de configuració (ex: actualitzar el linter).

Exemple de commit traçable:

feat(auth): afegir autenticació amb Google en el login

S'ha implementat el flux d'OAuth2 utilitzant la llibreria oficial.
Això permetrà reduir la fricció en el registre d'usuaris.

Resol incidència: #432

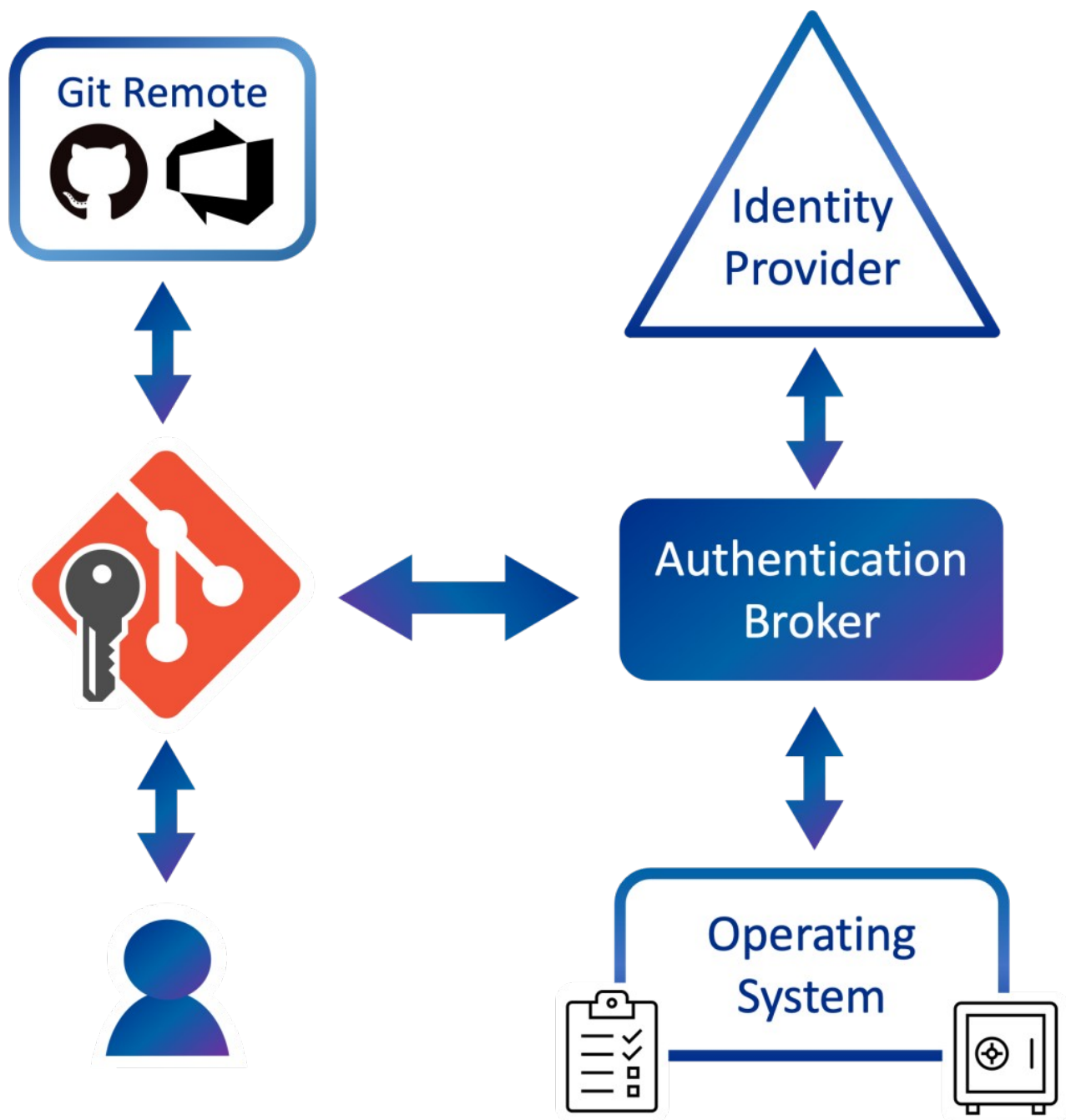
Aquest missatge és traçable perquè especifica l'àmbit (auth), explica el motiu en el cos i enllaça directament amb el tiquet #432 de l'eina de gestió de projectes.



Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un repositori local en Git.
2. Configurar les regles de protecció d'una branca simulada.
3. Crear una branca feature/login, fer dos commits seguint l'estàndard *Conventional Commits* i fusionar-la simulant una revisió de codi.

TEMA 2: Autenticació, Autorització i Governança en Repositoris



- 1 Conceptes de Governança en Plataformes de Codi Font
 - 2 Mecanismes d'Autenticació Segura
 - 3 Seguretat de la Identitat: Firmes GPG i SSH en els Commits
 - 4 Integració d'Eines i Automatismes mitjançant Credencials de Sistema
 - 5 Regles de Protecció de Branques (Branch Protection Rules)
- 🔧 Pràctica de Laboratori Proposada per a aquest tema

1 Conceptes de Governança en Plataformes de Codi Font

La governança en un repositori de codi és el conjunt de regles, polítiques i processos que determinen com s'accedeix, es modifica i es protegeix el codi font d'una organització. En entorns de **Integració Contínua (CI)**, la governança assegura que cap codi pugui arribar a producció sense haver sigut auditat i autoritzat, evitant, per exemple, injeccions de codi maliciós.

Les plataformes principals (GitHub, GitLab, Bitbucket) apliquen el model **RBAC (Role-Based Access Control)** o Control d'Accés Basat en Rols.

Rols típics en plataformes de codi:

- **Owner / Administrator:** Control total del repositori. Pot esborrar-lo, canviar la visibilitat (públic/privat) i gestionar els mètodes de pagament i seguretat.
- **Maintainer / Developer:** Pot escriure directament en branques no protegides, crear *Pull Requests*, etiquetar versions i gestionar incidències.
- **Reporter / Viewer:** Accés de només lectura. Ideal per a auditors o membres de l'equip que només necessiten consultar el codi o reportar errors (*bugs*).

2 Mecanismes d'Autenticació Segura

Per a interactuar amb un repositori remot (fer `git push` o `git pull`), s'han d'utilitzar mecanismes d'autenticació robustos que eliminin l'ús de contrasenyes tradicionals en text pla.

A) Claus SSH (Secure Shell)

Basades en criptografia de clau pública/privada. El desenvolupador genera un parell de claus a la seua màquina:

1. La **clau privada** es queda de forma segura a l'ordinador local.
2. La **clau pública** es puja a la plataforma (GitHub/GitLab).
*En l'actualitat es recomana l'algorisme **ED25519** per sobre d'**RSA** per ser més segur i eficient.*

B) Tokens d'Accés Personal (PAT - Personal Access Tokens)

Substitueixen les contrasenyes quan s'utilitza el protocol HTTPS.

- **Principi de mínim privilegi:** Un PAT es pot configurar perquè només tinga permisos de lectura d'un repositori concret, caduque en 30 dies i no pugui fer res més.
- Són essencials per a connectar les eines de CI (com Jenkins o runners externs) amb el repositori.

3 Seguretat de la Identitat: Firmes GPG i SSH en els *Commits*

En Git, qualsevol usuari pot configurar el seu entorn local amb el nom i correu de qualsevol altra persona (per exemple, executant `git config user.name "Linus Torvalds"`). Això obri la porta a la suplantació d'identitat.

Per a evitar-ho, les directrius de seguretat digital exigeixen la **firma criptogràfica de commits**:

- El desenvolupador utilitza una clau **GPG (GNU Privacy Guard)** o una clau **SSH** per a firmar digitalment cada commit.
- La plataforma remota verifica la firma contra la clau pública de l'usuari.
- Si coincideix, mostra un distintiu visual de **"Verified" (Verificat)**. Si el pipeline de CI troba un commit sense verificar en una branca crítica, pot avortar el procés automàticament.

4 Integració d'Eines i Automatismes mitjançant Credencials de Sistema

Quan un servidor de CI (com GitHub Actions o GitLab CI) necessita descarregar el codi per a compilar-lo, no utilitza el compte d'un programador humà. S'utilitzen **comptes de servei** o credencials temporals injectades en temps d'execució.

Bones pràctiques en la integració de credencials:

1. **Injecció de Secrets com a Variables d'Entorn:** Les claus d'API, contrasenyes de bases de dades de test i tokens de desplegament mai s'han d'escriure en el codi font (*hardcoding*). Es configuren en l'apartat de *Secrets* de la plataforma de CI i s'injecten de forma oculta durant l'execució del pipeline.
2. **Secrets emmascarats:** Les eines de CI modernes detecten si un secret s'intenta imprimir en la consola de depuració (*logs*) i el substitueixen automàticament per asteriscs (*******).

5 Regles de Protecció de Branques (*Branch Protection Rules*)

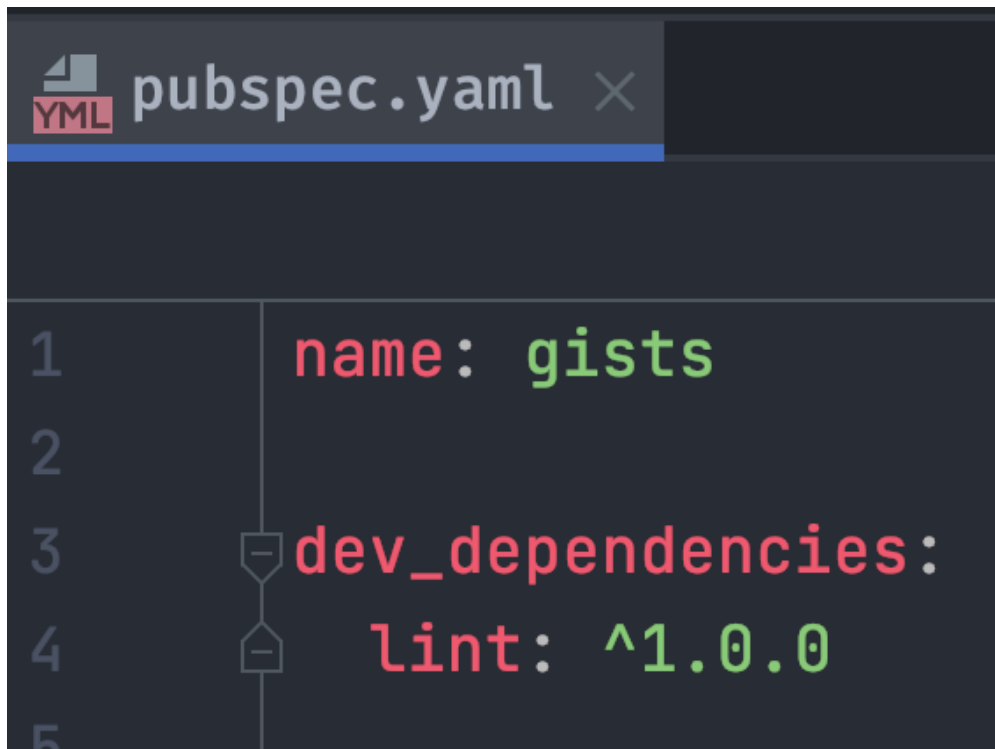
Aquest és el punt clau on la governança es fusiona amb l'automatització de la CI. Per a protegir les branques estables (main o release), es configuren les següents regles obligatòries:

- **Inhabilitar el *Push* directe:** Ningú (ni tan sols els administradors, idealment) pot fer canvis directament sobre la branca. Qualsevol modificació ha d'entrar mitjançant una **Pull Request (PR)** o **Merge Request (MR)**.
- **Revisió per parells (*Code Review*):** Exigir que almenys un o dos desenvolupadors aproven els canvis manualment després de revisar el codi abans que es permeti la fusió.
- **Verificació d'estat requerida (*Required Status Checks*):** La branca es bloqueja i no permet la fusió si el pipeline d'Integració Contínua ha fallat (per exemple, si un test no ha passat o el linter ha detectat errors).
- **Historial lineal:** Exigir combinacions tipus *Squash merge* o *Rebase* per a mantindre un històric de commits net i fàcil de seguir.

 **Pràctica de Laboratori Proposada per a aquest tema:**

1. Configurar una clau SSH (ED25519) i enllaçar-la amb un compte de GitHub o GitLab.
2. Crear un repositori privat, anar a la configuració de branques i activar una regla de protecció per a main que impedisca fer *push* directe i exigisca que el pipeline de CI (crearem un de bàsic en el bloc 2) acabe correctament.

TEMA 3: Validació Automatitzada de Codi (Linterització i Dependències)



```
1  name: gists
2
3  dev_dependencies:
4    lint: ^1.0.0
5
```

- 1 Introducció a l'Anàlisi Estàtica de Codi
 - 2 Ús de Linters i Eines d'Avaluació Sintàctica i Semàntica
 - 3 Gestió de Llibreries Externes i Genèriques
 - 4 Comprovació i Auditoria Automatitzada de Versions
 - 5 Directives de Qualitat: El Concepte de Quality Gates (Portes de Qualitat)
- 🔧 Pràctica de Laboratori Proposada per a aquest tema:

1 Introducció a l'Anàlisi Estàtica de Codi

L'anàlisi estàtica de codi és el procés d'avaluar el codi font d'una aplicació sense arribar a executar-lo. Dins d'un flux d'Integració Contínua (CI), aquesta és la primera línia de defensa. Es col·loca al principi del *pipeline* perquè és una operació molt ràpida que permet detectar errors estructurals, de sintaxi o d'estil abans de gastar recursos compilant o llançant tests complexos.

Aquest procés garanteix que el codi s'ajuste a les directives de qualitat de l'organització, assegurant que tot l'equip programe amb el mateix estil (*homogeneïtat del codi*).

2 Ús de *Linters* i Eines d'Avaluació Sintàctica i Semàntica

Un **Linter** és una eina de programari que analitza el codi i llança alertes sobre construccions que, tot i ser vàlides per al llenguatge, podrien causar errors en el futur o dificultar-ne el manteniment.

Funcions principals dels Linters:

- **Validació Sintàctica:** Detectar errors de text, com ara parèntesis no tancats, variables declarades però mai utilitzades o crides a funcions inexistents.
- **Format i Estil:** Imposar regles sobre l'ús d'espais o tabuladors, la longitud màxima d'una línia o l'ús de cometes simples vs. dobles.
- **Evitació de *Bad Practices* (Males pràctiques):** Identificar l'ús de funcions obsoletes (*deprecated*) o estructures de control excessivament complexes (alta complexitat ciclomàtica).

Exemples de Linters segons el llenguatge:

- **JavaScript / TypeScript:** ESLint, Prettier.
- **Python:** Flake8, Black, Pylint.
- **Java:** Checkstyle, PMD.
- **HTML / CSS:** HTMLHint, Stylelint.

3 Gestió de Llibreries Externes i Genèriques

El programari modern rarament es programa des de zero; depèn en gran mesura de llibreries externes o paquets de tercers (coneguts com a *dependències*). Cada llenguatge té el seu propi Gestor de Dependències per a descarregar, actualitzar i controlar aquestes llibreries.

Llenguatge	Gestor de Dependències	Fitxer de Manifest (Configuració)	Fitxer de Bloqueig (<i>Lockfile</i>)
Node.js	npm / yarn	<u>package.json</u>	<u>package-lock.json</u> / <u>yarn.lock</u>
Python	pip / Poetry	<u>requirements.txt</u> / <u>pyproject.toml</u>	<u>poetry.lock</u>
Java	Maven / Gradle	<u>pom.xml</u> / <u>build.gradle</u>	(<i>Depèn de la configuració de</i>

			<i>plugins)</i>
PHP	Composer	<u>composer.json</u>	<u>composer.lock</u>

El paper del Fitxer de Bloqueig (Lockfile) en la CI:

Un dels errors més comuns en CI és que un codi funcione en la màquina del programador però falle en el servidor de construcció. Per a evitar-ho, **és obligatori incloure el fitxer de bloqueig en el repositori**. Aquest fitxer guarda la versió exacta i el *hash* de seguretat de cada subdependència instal·lada, garantint que el pipeline de CI construeixca l'aplicació exactament amb les mateixes llibreries que va utilitzar el desenvolupador.

4 Comprovació i Auditoria Automatitzada de Versions

Amb el pas del temps, les llibreries externes es queden obsoletes. Això pot provocar problemes d'incompatibilitat o pèrdua de rendiment. En el flux de CI, s'integren eines automatitzades que s'encarreguen de:

1. **Detecció de dependències obsoletes:** Analitzar si hi ha versions més recents de les llibreries que utilitza el nostre projecte (ex: executant `npm outdated` en Node.js).
2. **Automatització d'actualitzacions (Bots de CI):** Eines com **Dependabot** (natiu de GitHub) o **Renovate** es configuren per a revisar el repositori de manera periòdica. Si detecten una llibreria obsoleta, creen automàticament una *Pull Request* actualitzant el fitxer de manifest. El pipeline de CI s'executa sobre eixa PR per a comprovar que la nova versió de la llibreria no trenca el codi existent.

5 Directives de Qualitat: El Concepte de Quality Gates (Portes de Qualitat)

Dins del servidor d'Integració Contínua, l'anàlisi estàtica es pot connectar a plataformes centralitzades de qualitat com **SonarQube** o **SonarCloud**. Aquestes eines defineixen el que s'anomena una **Quality Gate**.

Una *Quality Gate* és un conjunt de condicions Booleanes que el codi ha de complir obligatòriament per a poder ser fusionat en la branca estable. Si el codi no les compleix, el pipeline de CI es para i es marca com a fallit.

Exemples de criteris en una Quality Gate:

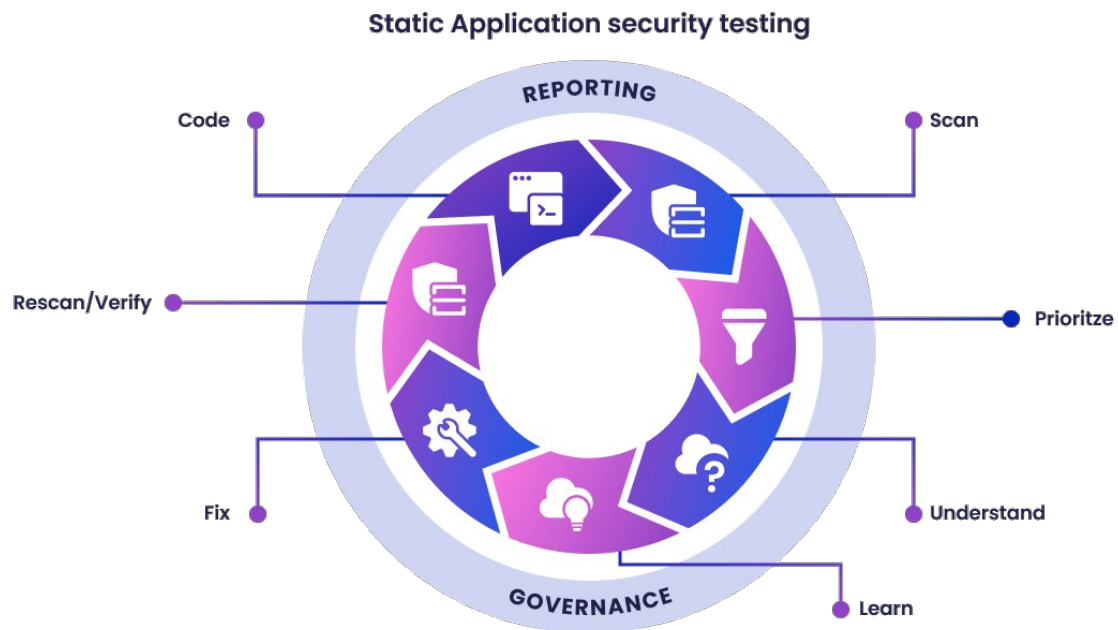
- Zero errors de sintaxi o *bugs* crítics detectats per l'analitzador.
- El percentatge de "Codi Duplicat" (*Code Duplication*) ha de ser inferior al 3%.
- El codi ha de complir un nivell mínim de mantenibilitat (Grau A o B).

Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un xicotet projecte (per exemple, en Python o Node.js) que continga variables no utilitzades i errors d'estil a propòsit.

2. Configurar un linter local (com eslint o flake8) i executar-lo per consola per a veure els errors.
3. Crear un fitxer de configuració del linter per a forçar regles estrictes de l'equip.
4. Executar una comprovació de dependències obsoletes utilitzant el gestor de paquets corresponent.

TEMA 4: Seguretat del Codi i Gestió de Vulnerabilitats (SAST)



- 1 Introducció al model DevSecOps i el concepte *Shift Left*
 - 2 Proves de Seguretat d'Aplicacions Estàtiques (SAST)
 - 3 Detecció de Secrets i Fuites d'Informació (Secret Scanning)
 - 4 Anàlisi de Composició de Programari (SCA)
- 🔧 Pràctica de Laboratori Proposada per a aquest tema:

1 Introducció al model DevSecOps i el concepte *Shift Left*

En el desenvolupament de programari tradicional, la seguretat es revisava al final del projecte, just abans del llançament. En la cultura DevOps moderna, aquest enfocament evoluciona cap al **DevSecOps**, on la seguretat s'integra de forma automatitzada des del primer moment en el flux de treball.

Això es basa en el principi de **Shift Left (Desplaçament a l'esquerra)**: consisteix a moure les proves de seguretat cap a les fases inicials del cicle de vida del desenvolupament (a l'esquerra del cronograma). Com més prompte es detecte una vulnerabilitat (en la màquina del programador o en la Integració Contínua), més fàcil i barat serà solucionar-la.

2 Proves de Seguretat d'Aplicacions Estàtiques (SAST)

Les eines **SAST (Static Application Security Testing)** analitzen el codi font, els binaris o els fitxers de configuració en busca de debilitats de seguretat conegudes sense executar l'aplicació.

Com funcionen les eines SAST?

Analitzen l'estructura del codi mitjançant un Arbre de Sintaxi Abstracta (AST) i segueixen el flux de les dades d'entrada des que l'usuari les introdueix fins que s'utilitzen (*Taint Analysis*). Això permet detectar si una dada "bruta" arriba a un lloc perillós sense haver sigut validada prèviament.

Vulnerabilitats comunes que detecta el SAST:

- **Injeccions de codi:** Injecció SQL, injecció de comandes de sistema o Cross-Site Scripting (XSS).
- **Gestió de memòria insegura:** Desbordaments de memòria intermèdia (*buffer overflow*) en llenguatges com C/C++.
- **Criptografia dèbil:** Ús d'algorismes de xifratge obsolets o trencats (com MD5 o SHA-1) per a guardar contrasenyes.
- **Configuracions de seguretat incorrectes:** Permisos excessius en fitxers o comunicacions sense xifrar (HTTP en lloc d'HTTPS).

Eines SAST de referència:

- **SonarQube / SonarCloud:** Plataforma generalista que inclou un motor de seguretat SAST molt potent.
- **Semgrep:** Un motor d'anàlisi estàtica molt ràpid i configurable mitjançant regles YAML.
- **CodeQL:** L'eina nativa que utilitza GitHub Advanced Security per a fer consultes sobre el codi com si fos una base de dades.

3 Detecció de Secrets i Fuites d'Informació (*Secret Scanning*)

Un dels errors de seguretat més greus i comuns en els repositoris de codi és el **Hardcoding de secrets**: incloure contrasenyes, claus d'API de serveis (AWS, Google Maps, Stripe) o claus privades directament en el text del codi.

Si el repositori es fa públic per error, o un atacant aconsegueix accés, aquestes credencials queden exposades de forma immediata.

Com actua la CI contra la fuga de secrets?

S'integren eines específiques que es dediquen a escanejar l'historial complet de *commits* utilitzant expressions regulars (Regex) i anàlisi d'entropia (per a detectar cadenes de text aleatòries i llargues que semblen contrasenyes).

- **Eines populars:** TruffleHog, GitLeaks o GitGuardian.
- **Polítiques de bloqueig:** Es poden configurar en el flux de CI com a un pas obligatori. Si es detecta una cadena que coincideix amb el patró d'una clau privada, el *pipeline* llança una alerta i bloqueja la integració de codi.

4 Anàlisi de Composició de Programari (SCA)

Mentre que el SAST busca errors en el codi escrit per l'equip, l'**SCA (Software Composition Analysis)** busca vulnerabilitats conegudes en les **llibries de tercers** que utilitza el projecte (estudiades en el Tema 3).

El procés d'auditoria SCA:

1. L'eina llegeix el fitxer de manifest del projecte (`package.json`, `requirements.txt`, etc.).
2. Elabora un inventari de totes les dependències directes i indirectes (transitives).
3. Comprova aquest catàleg contra bases de dades públiques de vulnerabilitats, com la **NVD (National Vulnerability Database)** dels EUA o el registre de **CVE (Common Vulnerabilities and Exposures)**.
4. Si troba una llibreria amb una vulnerabilitat detectada, notifica la seua gravetat segons l'escala **CVSS (Common Vulnerability Scoring System)** (Baixa, Mitjana, Alta, Crítica).

Eines SCA populars:

- **OWASP Dependency-Check:** Eina gratuïta i de codi obert molt utilitzada en pipelines de Jenkins.
- **Snyk:** Una de les plataformes comercials més potents per a analitzar dependències i contenidors.
- **GitHub Dependabot alerts:** Escaneja de forma nativa i gratuïta les vulnerabilitats en els repositoris de GitHub.

Pràctica de Laboratori Proposada per a aquest tema:

1. Descarregar i configurar localment l'eina de codi obert **GitLeaks** o **TruffleHog**.

2. Crear un fitxer de text en un repositori de proves i escriure una clau d'AWS simulada (per exemple:
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY).
3. Executar l'eina per consola i verificar com detecta la credencial exposada.
4. Integrar l'eina semgrep de forma local per a passar un escaneig de seguretat bàsic sobre un fitxer de codi que continga una funció vulnerable.

TEMA 5: Manteniment de Repositoris a Gran Escala



- 1 Reptes en la Gestió de Repositoris de Gran Tamany
- 2 Estratègies i Tecnologies d'Optimització
- 3 Processos Periòdics de Còpia de Seguretat (*Backup*)
- 4 Recuperació de Desastres (*Disaster Recovery*) de l'Històric de Codi
- 🔧 Pràctica de Laboratori Proposada per a aquest tema

1 Reptes en la Gestió de Repositoris de Gran Tamany

Quan un projecte de programari creix i suma anys d'històric, centenars de desenvolupadors i milers de commits, els repositoris de Git poden patir degradacions de rendiment importants. Aquests megarepositoris o estratègies de **Monorepo** (un sol repositori per a tota l'organització) s'enfronten a tres grans problemes:

- **Operacions de Git lentes:** Ordres diàries com `git clone`, `git checkout` o `git status` triguen minuts en lloc de mil·lisegons a causa de la quantitat d'objectes i fitxers a analitzar.
- **Bloqueig del servidor de CI:** El motor d'Integració Contínua tarda massa temps a descarregar el codi font per a cada execució del pipeline, saturant l'amplada de banda i la memòria.
- **Fitxers binaris pesants:** L'emmagatzematge d'imatges, vídeos, arxius de configuració compilats o dependències dins de Git (que no està dissenyat per a binaris) infla el tamany del directori `.git`.

2 Estratègies i Tecnologies d'Optimització

Per a garantir que el flux de treball es mantinga àgil, les directives d'operació exigeixen l'aplicació de tècniques d'optimització d'infraestructura de codi:

A) Git LFS (Large File Storage)

És una extensió de codi obert per a Git que substitueix els fitxers grans (com gràfics, binaris o bases de dades de prova) per **punters de text** dins del repositori. El fitxer real es guarda en un servidor d'emmagatzematge separat.

- **Impacte en la CI:** Quan el servidor de CI fa un clonat, només descarrega els punters de text, estalviant temps i espai, tret que s'indique explícitament que es necessiten els binaris per als tests.

B) Clonat parcial i clonat superficial (*Shallow Clone*)

Configurar els agents del pipeline de CI per a descarregar només el que és estrictament necessari per a validar el codi:

- `--depth 1`: Limita el clonat a l'**últim commit** de la branca, ignorant tot l'històric d'anys enrere. És l'estàndard per a la majoria de pipelines de construcció ràpids.
- Sparse Checkout (Saturació dispersa): Permet al sistema de CI descarregar només carpetes o directoris concrets d'un repositori gegant, ideal per a arquitectures de monorepositoris on només s'ha modificat un microservei.

3 Processos Periòdics de Còpia de Seguretat (*Backup*)

Tot i que Git és distribuït i cada desenvolupador en té una còpia, el servidor central (GitHub, GitLab, etc.) conté metadades crítiques que no formen part del repositori Git bàsic: **Pull Requests, comentaris de revisió de codi, claus d'accés, secrets de CI i configuració de pipelines.**

Les directives de seguretat digital exigeixen automatitzar còpies de seguretat periòdiques d'aquesta infraestructura centralitzada:

1. **Automatització de Backups:** Programació de tasques periòdiques (*cron jobs*) que utilitzen les APIs oficials de les plataformes per a exportar tant el codi font com les metadades de gestió.
2. **Emmagatzematge immutable i descentralitzat:** Guardar aquestes còpies en un proveïdor d'emmagatzematge al núvol (com AWS S3 o Azure Blob) totalment aïllat del sistema principal, utilitzant polítiques de retenció de dades obligatòries (per exemple, conservar backups diaris durant 30 dies).

4 Recuperació de Desastres (*Disaster Recovery*) de l'Històric de Codi

El pitjor escenari per a un enginyer DevOps és la pèrdua de traçabilitat, la corrupció de la base de dades del repositori central o un atac de *ransomware* als servidors de codi.

Un pla de recuperació de desastres eficaç ha d'incloure:

- **Mesurament del RPO (Recovery Point Objective):** El temps màxim de feina que l'empresa es pot permetre perdre (ex: menys de 24 hores de codi gràcies a backups diaris).
- **Mesurament del RTO (Recovery Time Objective):** El temps que es triga a restaurar el servei perquè els programadors puguin tornar a treballar i la CI torne a funcionar.
- **Estratègia de restauració automatitzada:** Guions (*scripts*) provats periòdicament que agafen la còpia de seguretat de l'emmagatzematge aïllat, recreen l'estructura de rols i usuaris, i tornen a publicar els repositoris en una plataforma alternativa si la principal ha caigut (per exemple, moure el flux de GitHub a un servidor GitLab d'emergència).

Pràctica de Laboratori Proposada per a aquest tema:

1. Instal·lar i configurar **Git LFS** en un repositori local. Configurar-lo per a fer el seguiment de fitxers .zip o .mp4 mitjançant l'ordre git lfs track "*.zip".
2. Simular un procés de CI executant de forma manual una ordre de clonat superficial (git clone --depth 1 <url>) i comprovar de quina manera es redueix la mida de la carpeta .git descarregada.
3. Crear un *script* automatitzat en Bash o Python que utilitzi l'eina de línia de comandes de la plataforma (com gh per a GitHub) per a descarregar un fitxer comprimit amb el contingut del repositori com a còpia de seguretat simulada.

TEMA 6: Automatització i Creació de Serveis via CLI i API



- 1 El Rol de l'Automatització en els Fluxos de CI
 - 2 Interfícies de Línia de Comandes (CLI) en la CI
 - 3 Interfícies de Programació d'Aplicacions (API) en la CI
 - 4 Automatismes mitjançant Llenguatges de Programació i *Scripting*
- 🔧 *Pràctica de Laboratori Proposada per a aquest tema*

1 El Rol de l'Automatització en els Fluxos de CI

Dins de la filosofia DevOps, qualsevol intervenció manual per a configurar entorns o crear serveis es considera un vector d'error, un coll de botella i una pèrdua de traçabilitat. L'**automatització absoluta** garanteix que el camí que segueix el codi des de la màquina del desenvolupador fins a la creació dels serveis necessaris siga **repetible, predictable i auditable**.

Per a aconseguir que el servidor d'Integració Contínua interactue de manera autònoma amb plataformes al núvol, servidors i orquestradors, s'utilitzen principalment dues interfícies: les **CLIs** (Interfícies de Línia de Comandes) i les **APIs** (Interfícies de Programació d'Aplicacions).

2 Interfícies de Línia de Comandes (CLI) en la CI

Una **CLI (Command Line Interface)** és una eina que permet executar instruccions en el sistema operatiu mitjançant text escrit. En els *pipelines* de CI (com els fitxers YAML de GitHub Actions o GitLab CI), els passos d'execució (*steps*) són essencialment crides a la línia de comandes d'eines CLI instal·lades en els agents o *runners*.

CLIs essencials en entorns de contenidors i CI:

- **CLIs de Cloud Providers:** Eines com `aws-cli`, `gcloud` (Google Cloud) o `az` (Azure) que permeten crear bases de dades, xarxes o balancejadors de càrrega de manera immediata.
- **CLI d'Orquestració i Contenedors:** `docker` (per a construir i pujar imatges) i `kubect` (per a gestionar de manera directa els clústers de Kubernetes).
- **CLIs de Control de Versions:** Eines com `gh` (GitHub CLI) o `glab` (GitLab CLI) que permeten gestionar el propi repositori, aprovar *Pull Requests* o descarregar berts des de la terminal.

3 Interfícies de Programació d'Aplicacions (API) en la CI

Quan les necessitats d'automatització superen el que una CLI pot oferir de manera directa, s'utilitzen les **APIs (Application Programming Interfaces)**. La immensa majoria de plataformes modernes ofereixen APIs de tipus **REST** (basades en HTTP i intercanvi de dades en format **JSON**) o **gRPC**.

El servidor de CI realitza peticions HTTP (`GET`, `POST`, `PUT`, `DELETE`) cap als *endpoints* de la plataforma per a:

- Consultar l'estat d'un desplegament.
- Crear de manera dinàmica nous usuaris o assignar permisos temporals.
- Sol·licitar la creació de recursos de computació a demanda en el proveïdor de núvol.

L'autenticació en aquestes APIs es realitza de manera segura mitjançant els mecanismes estudiats en el Tema 2, com ara els **Tokens d'Accés Personal (PAT)** o l'intercanvi de claus temporals (OIDC).

4 Automatismes mitjançant Llenguatges de Programació i *Scripting*

Per a connectar les CLIs i les APIs en lògiques complexes (gestió d'errors, bucles, estructures condicionals o tractament de dades JSON), s'utilitzen automatismes desenvolupats en dos entorns:

A) Scripts de Shell (Bash / Zsh)

- **Característiques:** Són directes, ràpids i estan disponibles de manera nativa en pràcticament qualsevol agent de CI basat en Linux.
- **Ús típic:** Execució seqüencial de comandes, verificació de codis de retorn (exit 0 o exit 1) i redirecció de fluxos de text.

B) Llenguatges de Programació (Python / Go)

- **Característiques:** Es prefereixen quan la lògica de l'automatisme és complexa. Python és l'estàndard industrial per a *scripting* avançat gràcies a la seua llegibilitat i potència de llibreries.
- **Ús típic:** Utilització de llibreries com requests per a interactuar amb APIs REST, o **SDKs oficials** (com boto3 per a AWS), que permeten gestionar la infraestructura de manera nativa com a objectes del llenguatge de programació.

5 Creació Automatitzada de Serveis Requerits

El resultat final d'aplicar CLI, API i *scripting* és la capacitat d'executar el cicle de vida complet d'un servei sense intervenció humana:

1. **Aixecament de dependències en fase de test:** El pipeline de CI utilitza un script per a aixecar una base de dades efímera (per exemple, un contenidor PostgreSQL de prova), importar un esquema de dades bàsic i deixar el servei llest perquè l'aplicació hi passe els tests d'integració.
2. **Neteja d'entorns (Teardown):** Una vegada finalitzades les proves, l'automatisme s'encarrega de destruir els serveis creats per a no consumir recursos de la infraestructura d'orígens.

Pràctica de Laboratori Proposada per a aquest tema:

1. Instal·lar la CLI oficial de la plataforma de codi que estigues utilitzant (per exemple, gh per a GitHub).
2. Autenticar la CLI utilitzant un token d'accés (PAT) generat amb els permisos mínims.
3. Crear un script en Bash o un script en Python (utilitzant la llibreria requests) que realitzi una petició a l'API de la plataforma per a **crear de manera automàtica una nova incidència (issue)** amb el títol "Error detectat en la build automatitzada".
4. Executar l'script i comprovar que la incidència apareix immediatament en la plataforma web.

TEMA 7: Parametrització d'Artefactes en el Cicle de Vida



- 1 El Concepte d'Artefacte i el seu Cicle de Vida
 - 2 El Principi de Separació de Codi i Configuració
 - 3 Parametrització Orientada a la Gestió de la Configuració
 - 4 Integració de Versions de Dades de les Aplicacions
 - 5 Reutilització d'Artefactes i Atributs Únics
-  Pràctica de Laboratori Proposada per a aquest tema

1 El Concepte d'Artefacte i el seu Cicle de Vida

En el context de la Integració Contínua, un **artefacte** és qualsevol fitxer o conjunt de fitxers generat durant el procés de construcció (*build*) que està llest per a ser provat, emmagatzemat o desplegat. Exemples d'artefactes són un arxiu binari (com un fitxer `.jar` en Java o un executable en Go), un paquet de llibreries (paquet `.npm` o `.tar.gz`) o, de manera molt comuna, una **imatge de contenidor** (Docker/Podman).

El cicle de vida d'un artefacte inclou la seua creació automatitzada, la seua parametrització, el seu emmagatzematge en un registre centralitzat (com un *Artifact Registry* o Docker Hub) i la seua distribució cap als diferents entorns.

2 El Principi de Separació de Codi i Configuració

Una de les directrius d'operació i qualitat més importants (recollida en la metodologia dels *Dotze Factors* o *12-Factor App*) és **separar estrictament la configuració del codi font**.

El codi font de l'aplicació ha de ser exactament el mateix de camí a tots els entorns (desenvolupament, integració, preproducció i producció). El que canvia entre un entorn i un altre és la **configuració** (les credencials de la base de dades, els tokens de tercers, els ports d'escolta, etc.).

Si canvia l'entorn, no es torna a compilar el codi, simplement es canvia la **parametrització de l'artefacte** a través de:

- **Variables d'entorn:** Variables que el sistema operatiu o l'orquestrador injecten en el contenidor en temps d'execució.
- **Fitxers de configuració externs:** Fitxers (com `.env`, JSON o YAML) que es munten de manera dinàmica dins del servei.

3 Parametrització Orientada a la Gestió de la Configuració

Quan el pipeline de CI gestiona el cicle de vida dels serveis de l'aplicació, ha de controlar com es parametritzen els artefactes en funció del programari. Això inclou la **gestió de la configuració de les aplicacions** (*Application Configuration Management*).

- **Injecció de paràmetres dinàmics:** El pipeline utilitza plantilles de configuració on substitueix variables en temps de construcció o desplegament (per exemple, canviar la URL d'una API de l'entorn de proves per la de l'entorn de producció).
- **Control de versions de configuració:** La mateixa configuració es tracta com a codi (sovint en un repositori separat de Git, seguint pràctiques de *GitOps*), assegurant que qualsevol canvi en un paràmetre quede registrat, auditat i associat a una versió concreta de l'aplicació.

4 Integració de Versions de Dades de les Aplicacions

Les aplicacions no funcionen de manera aïllada, depenen estretament de bases de dades o estructures d'emmagatzematge. La parametrització d'un artefacte en la CI també ha de considerar com s'integren les **versions de les dades**.

- **Migracions de Base de Dades Automatitzades:** El pipeline de CI integra eines (com *Flyway*, *Liquibase* o els ORMs del propi llenguatge com Django Migrations o Prisma) per a aplicar canvis en l'esquema de la base de dades de forma automàtica.
- **Parametrització de scripts de dades:** S'utilitzen paràmetres per a indicar a l'aplicació quina versió del *schema* de dades necessita o per a executar scripts de càrrega de dades simulades (*seeding*) en els entorns de test automàtic, assegurant que les dades evolucionen al mateix ritme que el programari.

5 Reutilització d'Artefactes i Atributs Únics

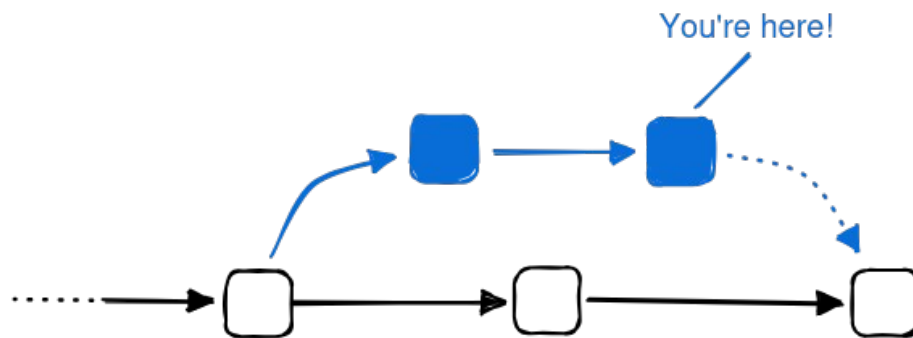
Perquè un artefacte o configuració de servei siga **reutilitzable** en el futur o en diferents contextos, la seua definició ha de ser modular i abstracta. No obstant això, per a evitar col·lisions en entorns compartits, s'han de definir paràmetres específics que garantisquen que determinats elements siguin únics:

- **Nom del servei:** S'utilitzen prefixos o sufixos dinàmics basats en la branca de Git o el número de build (ex: servei-auth-pr-42).
- **Regió geogràfica:** Paràmetres que defineixen on es desplegarà el recurs al núvol (ex: eu-west-1 o us-east-1) per a complir directrius de latència o de seguretat digital (com el RGPD).
- **Recursos assignats:** Definició modular de límits de CPU, memòria RAM i emmagatzematge (ex: limits.memory: "512Mi").
- **Permisos:** Assignació de rols de seguretat específics (com rols IAM de núvol) necessaris per a eixe servei concret.

Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un xicotet fitxer de configuració d'una aplicació (per exemple, un config.json o un fitxer .env) on les credencials i les URLs estiguen buides o continguin variables de substitució (ex: \${DB_HOST}).
2. Crear un script en Bash o utilitzar una eina de substitució de plantilles (com envsubst) que agafe variables d'entorn del sistema de CI i genere el fitxer de configuració final parametritzat de manera automàtica.
3. Comprovar que, canviant el valor de la variable d'entorn en la terminal, el fitxer final es modifica correctament sense haver de tocar una sola línia del codi de l'aplicació.

TEMA 8: Infraestructura com a Codi (IaC) i Contenedors en la IC



- 1 El Concepte d'Infraestructura com a Codi (IaC)
 - 2 Definició d'Artefactes per a l'Automatització de la Infraestructura
 - 3 El Principi d'Idempotència
 - 4 Enfocament Declaratiu vs. Imperatiu en IaC
-  Pràctica de Laboratori Proposada per a aquest tema

1 El Concepte d'Infraestructura com a Codi (IaC)

La **Infraestructura com a Codi (IaC)** és la pràctica de gestionar i aprovisionar la infraestructura tecnològica (xarxes, servidors, balancejadors de càrrega, bases de dades) mitjançant **fitxers de configuració declaratius o scripts definits en codi font**, en lloc de fer-ho mitjançant processos manuals o interfícies gràfiques.

Dins d'un pipeline d'Integració Contínua, la IaC permet tractar la infraestructura exactament igual que el programari de l'aplicació: es guarda en el mateix repositori de Git (o en un dedicat), està subjecta a control de versions, passa per linters i es desplega de manera 100% automatitzada.

2 Definició d'Artefactes per a l'Automatització de la Infraestructura

Depenent de l'arquitectura i les directrius d'operació del sistema, els artefactes i plantilles responsables de definir l'entorn es poden agrupar en diferents tipus:

- **Contenidors:** El format de paquetització estàndard en DevOps. Es defineixen mitjançant plantilles declaratives (com un **Dockerfile**) on s'especifica la imatge base, les llibreries del sistema, el codi binari de l'aplicació i el comando d'arrancada.
- **Màquines Virtuals i Físiques:** S'utilitzen eines com **Packer** per a crear imatges de disc (AMIs d'AWS, imatges de VMware) parametritzades a partir d'un fitxer de configuració JSON o HCL, garantint que el sistema operatiu base estiga preconfigurat de forma idèntica.
- **Scripts i Codi Binari:** Guions en Bash, Ansible Playbooks, o binaris compilats d'automatització que s'encarreguen de tasques d'aprovisionament local o d'orquestració.

3 El Principi d'Idempotència

El criteri d'avaluació normatiu exigeix verificar que el codi font de la integració siga **idempotent**.

Definició d'Idempotència: És la propietat d'una operació o script que garanteix que, **no importa quantes vegades s'execute o es cride, el resultat final sempre serà el mateix**, deixant el sistema exactament en l'estat desitjat sense introduir efectes secundaris o duplicacions.

Per què és crucial en la Integració Contínua?

Si un pipeline de CI falla a meitat d'execució i s'ha de tornar a llançar des del principi, un script *no idempotent* (com un script Bash que conté `mkdir carpeta` o `apt-get install nginx` sense condicions) donarà un error perquè la carpeta ja existeix o intentarà reinstallar el servei trencant la configuració actual.

En canvi, un codi o plantilla declarativa idempotent (com els fitxers de configuració de **Terraform**, **Ansible** o **Kubernetes Manifests**):

1. Analitza l'estat actual del sistema.
2. El compara amb l'estat desitjat definit en el codi.
3. **Només executa els canvis necessaris** per a passar de l'estat actual a l'estat desitjat. Si el sistema ja està en l'estat desitjat, l'operació acaba correctament fent un *no-op* (cap acció).

4 Enfocament Declaratiu vs. Imperatiu en IaC

Per a aconseguir la idempotència de manera nativa, les eines modernes utilitzen un enfocament declaratiu:

- **Enfocament Imperatiu (El "Com"):** L'script detalla els passos concrets pas a pas que ha de fer el sistema. (Exemple: *"Aixeca una màquina virtual, després installa Docker, després descarrega la imatge, després fes un docker run..."*). És difícil de mantindre i fer idempotent de manera manual.
- **Enfocament Declaratiu (El "Què"):** El codi defineix com vols que quede el sistema, demanant a l'eina que s'encarregue d'executar les accions subjacents. (Exemple: *"Vull que hi haja un clúster amb 3 nodes i un contenidor web escoltant al port 80"*). Eines com **Terraform** o **Kubernetes YAMLs** són 100% declaratives.

Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un **Dockerfile** bàsic que empaquete una xicoteta aplicació web.
2. Escriure un script en Bash *no idempotent* per a descarregar un fitxer de text de proves i comprovar com, en llançar-lo per segona vegada, genera un error o duplica el fitxer.
3. Modificar l'script utilitzant condicions estructurals (com estructures if [! -f fitxer]; then ...) o reescriure el disseny utilitzant una eina declarativa/idempotent com una plantilla de configuració bàsica per a forçar que pugui executar-se 100 vegades consecutives donant sempre un resultat d'èxit.

TEMA 9: Execució de Pipelines de CI, Diagnòstic i Gestió de Fallades



- 1 Arquitectura d'un Motor d'Integració Contínua
 - 2 Resolució de Fallades i Proves de Diagnòstic
 - 3 Gestió de Dependències Externes en la Compilació
 - 4 Proporcionat d'Informació i Accions correctives
- 🔧 Pràctica de Laboratori Proposada per a aquest tema

1 Arquitectura d'un Motor d'Integració Contínua

Un **pipeline d'Integració Contínua** és la traducció en codi (normalment en formats declaratius com **YAML**) de tot el flux de treball que ha de patir el programari per a ser validat. L'arquitectura d'aquests sistemes (Jenkins, GitHub Actions, GitLab CI/CD) es basa en dos components:

- **El Servidor/Orquestrador Central:** Escolta els esdeveniments del repositori (com un *push* o una *Pull Request*) i coordina quines tasques s'han d'executar.
- **Els Agents / Runners / Workers:** Són les màquines (físiques, virtuals o, més comunament, contenidors efímers) on s'executen físicament les comandes del pipeline.

2 Resolució de Fallades i Proves de Diagnòstic

Quan el codi s'està integrant, el pipeline executa diferents nivells de **proves automatitzades** per a diagnosticar la salut de l'aplicació. Si qualsevol d'aquestes fases llança un codi d'error (exit code diferent de 0), el pipeline es deté immediatament per a evitar que el programari defectuós avanci en el cicle de vida.

Les eines integrades de diagnòstic avaluen quatre dimensions crítiques segons les directives de la normativa:

1. **Fallades d'Execució i Compilació:** S'avalua si el codi compila correctament o si té errors en temps d'execució inicial (*runtime*). S'executen els **tests unitaris** (que proven funcions aïllades) i els **tests d'integració** (que proven la interacció amb altres mòduls).
2. **Fallades de Qualitat:** Es comprova que no s'haja degradat la mantenibilitat de l'aplicació o que la cobertura de test (*Code Coverage*) no haja caigut per davall del llindar mínim admés (ex: un 80%).
3. **Fallades de Seguretat Digital:** Es verifiquen els resultats de les eines SAST i SCA (estudiades en el Tema 4). Si es detecta una vulnerabilitat de severitat alta o crítica, el pipeline es marca com a fallit.
4. **Fallades de Rendiment:** S'executen proves de càrrega o d'estrès automatitzades a xicoteta escala (utilitzant eines com *k6* o *Apache JMeter* integrades en línia de comandes) per a comprovar que els temps de resposta de les funcions es mantinguin dins dels límits acceptables.

3 Gestió de Dependències Externes en la Compilació

Un dels punts crítics en la build és la **verificació dels paràmetres del sistema que afecten la integració amb dependències externes** (com repositoris de paquets privats, dependències de xarxa o APIs de tercers).

Durant la compilació, el pipeline ha de:

- Configurar de manera segura les credencials d'accés als registres de paquets (ex: fitxers .npmrc per a npm o settings.xml per a Maven).

- Gestionar la memòria cau (*caching*) de les dependències. Descarregar gigabytes de llibreries d'Internet en cada build fa el procés lent i ineficient. Els sistemes de CI configuren paràmetres per a guardar en memòria cau aquestes carpetes de dependències, verificant mitjançant codis *hash* si han canviat abans de tornar-les a descarregar.

4 Proporcionat d'Informació i Accions correctives

Quan un pipeline falla, el sistema ha de proporcionar de manera immediata informació clara sobre els resultats i les accions correctives a l'equip de desenvolupament:

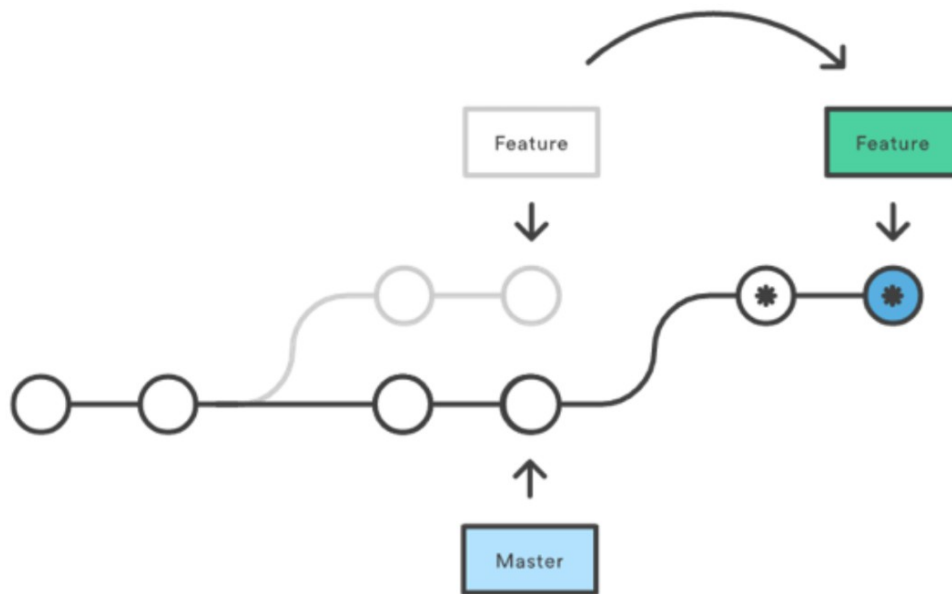
- **Logs estructurats:** Els entorns de CI agrupen les eixides de text per passos (*steps*) col·lapsables, facilitant la localització exacta de la línia de codi que ha provocat l'error.
- **Artefactes de diagnòstic:** Si un test d'interfície gràfica falla, el pipeline es pot configurar per a desar de forma automàtica captures de pantalla (*screenshots*) o vídeos de l'estat del sistema en eixe precís instant, publicant-los com a artefactes de diagnòstic descarregables.



Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un fitxer de configuració de pipeline bàsic (per exemple, un `.github/workflows/ci.yml` si s'utilitza GitHub o un `.gitlab-ci.yml` per a GitLab).
2. Definir tres fases seqüencials estructurades en el pipeline: `lint`, `build` i `test`.
3. Introduir un error de forma voluntària en els tests unitaris del projecte local (forçar un test que s'augmente que `1 == 2`).
4. Pujar el codi al repositori remot, observar com el pipeline s'executa, es deté en la fase de `test`, impedeix la integració del codi i genera l'informe de fallada amb els logs corresponents de diagnòstic.

TEMA 10: Integració de Codi, Resolució de Conflictes i Tancament del Flux



- 1 Publicació de Codi Validat en la Branca Estable
 - 2 Detecció i Resolució de Conflictes de Codi (*Merge Conflicts*)
 - 3 Auditoria de Canvis i Traçabilitat Temporal
 - 4 Tancament de l'Incidència en l'Eina de Seguiment
- 🔧 Pràctica de Laboratori Proposada per a aquest tema

1 Publicació de Codi Validat en la Branca Estable

Quan un desenvolupador ha finalitzat la seua tasca en una branca de treball (feature/*) i el pipeline d'Integració Contínua ha executat correctament totes les fases de validació (sintaxi, tests, seguretat i rendiment), el codi està preparat per a passar a la branca estable (main o production).

Aquest procés s'articula formalment mitjançant una **Pull Request (PR)** o **Merge Request (MR)**. La fusió final actua com a punt de control on es consolida el treball. Depenent de l'estratègia d'enramat de l'equip (Tema 1), s'utilitzaran diferents mètodes d'integració:

- **Merge tradicional:** Crea un commit de fusió que uneix l'historial de la branca de treball amb la branca principal.
- **Squash and Merge:** Agrupa tots els commits de la branca de treball en un sol commit net abans d'afegir-lo a la branca estable. És ideal per a mantindre un històric de producció llegible.
- **Rebase:** Reaplica els commits de la branca de treball directament sobre l'últim commit de la branca estable, mantenint un historial lineal.

2 Detecció i Resolució de Conflictes de Codi (*Merge Conflicts*)

En entorns de treball col·laboratiu, és molt habitual que dos o més desenvolupadors modifiquen de manera simultània les mateixes línies d'un mateix fitxer. Quan el primer fusiona els seus canvis, el repositori s'actualitza. Quan el segon intenta fusionar el seu codi, el sistema llança un **conflicte de fusió**.

Gestió de conflictes en el flux de CI:

1. **Notificació automàtica:** Les plataformes de codi bloquegen immediatament la possibilitat de fer la fusió i avisen el desenvolupador mitjançant la interfície de la PR.
2. **Resolució manual obligatòria:** El motor de CI no pot decidir quin codi és el correcte d'entre les dues variants. El desenvolupador ha de descarregar els últims canvis de la branca estable a la seua màquina local (git pull o git fetch + git rebase), obrir el fitxer afectat i resoldre el conflicte triant una de les opcions o combinant totes dues.
3. **Reverificació del pipeline:** Una vegada resolt el conflicte localment, es fa un nou git push. Això **dispara de nou el pipeline de CI** complet sobre la PR per a assegurar que la resolució manual del conflicte no ha trencat cap funcionalitat existent.

3 Auditoria de Canvis i Traçabilitat Temporal

Abans de donar per tancada la integració, les directives de qualitat i auditoria exigeixen comprovar rigorosament la traçabilitat de la modificació:

- **Comprovació de continguts modificats:** El sistema analitza els canvis línia per línia (*diff*) per verificar que només s'ha pujat el codi autoritzat i necessari per a resoldre la tasca, evitant la inclusió de fitxers brossa o temporals.
- **Verificació de dates de modificació i autoria:** Es corrobora la línia temporal dels commits per a garantir que no s'estan sobreescrivint versions més modernes amb codi antic (*code regression*).

4 Tancament de l'Incidència en l'Eina de Seguiment

L'últim pas en el cicle de vida del desenvolupament de programari és connectar l'èxit de la integració del codi amb la gestió de projectes. Les eines de CI es configuren per a automatitzar el **tancament de casos o incidències** de manera nativa.

- **Automatització per paraules clau:** Si en el missatge de commit o en la descripció de la Pull Request s'utilitzen paraules clau com Fixes #123, Closes #123 o Resolves #123, la plataforma (GitHub/GitLab) enllaçarà de manera automàtica el codi amb l'incidència número 123.
- **Tancament en la fusió:** En el precís instant en què la Pull Request s'aprova i es fusiona amb èxit a la branca estable, l'eina de seguiment canvia de forma automàtica l'estat d'eixa incidència a **"Done" (Fet) o "Closed" (Tancat)**. Això elimina la necessitat que els desenvolupadors actualitzen manualment els taulers de gestió (com Jira o Kanban) i assegura una sincronització perfecta entre l'estat real del codi i l'estat del projecte.



Pràctica de Laboratori Proposada per a aquest tema

1. Crear una incidència (*issue*) simulada en la teua plataforma de codi (GitHub/GitLab) amb el títol "Corregir text de la pàgina principal". Anota el número d'incidència assignat (per exemple, #1).
2. Crear una branca feature/fix-text, fer una modificació en un fitxer de text local i fer un commit que incloga el text: fix: corregir etiqueta de benvinguda (tanca #1).
3. Obrir una Pull Request cap a la branca main.
4. Forçar un conflicte de fusió modificant el mateix fitxer directament des de la interfície web de main.
5. Resoldre el conflicte en local, fer push i aprovar la Pull Request. Comprova com, en fusionar-se el codi, la incidència #1 es tanca de manera completament automàtica.

TEMA 11: Plataformes de Comunicació i Gestió de Projectes



- 1 La Cultura de la Transparència i Col·laboració en DevOps
- 2 Eines de Gestió de Projectes i Seguiment d'Incidències
- 3 Plataformes de Comunicació d'Equip (*Chat-Centric*)
- 4 El Concepte de ChatOps
-  Pràctica de Laboratori Proposada per a aquest tema

1 La Cultura de la Transparència i Col·laboració en DevOps

Un dels pilars fonamentals de la filosofia DevOps és la **destrucció de sils** entre els equips de desenvolupament (Dev) i operacions (Ops). Per a aconseguir un flux contínu i ràpid, la comunicació no pot dependre de correus electrònics aïllats o reunions esporàdiques. La infraestructura moderna exigeix l'ús de plataformes centralitzades on tota la informació sobre l'estat del projecte, els errors detectats i els canvis en el codi siga **transparent, accessible en temps real i auditable** per a qualsevol membre de l'equip.

2 Eines de Gestió de Projectes i Seguiment d'Incidències

Les eines de gestió de projectes en entorns de programari s'estructuren al voltant de **metodologies àgils** (com Scrum o Kanban). La unitat mínima de treball és la **incidència** o *ticket* (tasca, història d'usuari o error).

Característiques de les plataformes principals:

- **Jira (Atlassian):** L'estàndard industrial per a grans empreses. Permet una personalització total dels fluxos de treball (*workflows*), definint estats concrets per on ha de passar una tasca (ex: *To Do* ➡ *In Progress* ➡ *Code Review* ➡ *QA Testing* ➡ *Done*).
- **GitHub Projects / GitLab Issues:** Solucions natives integrades directament dins del propi repositori de codi. Ofereixen taulers Kanban visuals molt potents que faciliten enllaçar directament les tasques amb el codi font sense eixir de la plataforma.
- **OpenProject / Trello:** Alternatives de codi obert o més visuals, utilitzades per a equips que cerquen simplicitat o un control estricte de la privacitat de les seues dades en servidors propis (*on-premise*).

3 Plataformes de Comunicació d'Equip (*Chat-Centric*)

En equips DevOps, les eines de missatgeria han evolucionat de ser simples xats a convertir-se en **hubs operatius** on es concentra el dia a dia de l'enginyeria.

- **Slack:** Dissenyada específicament per a entorns de treball tècnics. Destaca per la seua capacitat d'organització en canals temàtics (ex: #alertes-produccio, #ci-builds, #desenvolupament-auth) i una potent API per a rebre automatitzacions.
- **Microsoft Teams:** Molt estesa en entorns corporatius i de l'ecosistema Microsoft. Combina canals de xat amb gestió de fitxers i videoconferències integrades.
- **Discord / Mattermost:** Utilitzades de manera habitual en comunitats de codi obert o empreses que requereixen una alternativa autogestionada (en el cas de Mattermost) per motius de seguretat digital.

4 El Concepte de ChatOps

El coneixement i ús d'aquestes plataformes és el pas previ per a implementar una de les pràctiques més avançades de col·laboració: el **ChatOps**.

Definició de ChatOps: És un model de treball que consisteix a connectar les persones, les eines de desenvolupament, els robots (*bots*) i la infraestructura a través d'una plataforma de comunicació centralitzada.

En lloc que un enginyer hagi d'entrar a una consola del núvol o a un servidor de CI aïllat per a executar una acció, escriu un comando directament en el xat de l'equip (ex: /deploy app to staging). Un bot escolta eixe comando, llança el pipeline de CI/CD corresponent, i retorna el resultat al mateix canal de xat perquè tot l'equip sàpiga què està passant en eixe moment.

Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un compte gratuït en una eina de gestió de projectes (com **GitHub Projects** dins del teu repositori d'estudis o un tauler bàsic de **Jira**).
2. Definir un flux de treball Kanban clàssic de 5 columnes: *Pendent*, *En Procés*, *Revisió de Codi*, *En Proves* i *Fet*.
3. Crear tres targetes de tasques simulades que representen funcionalitats de l'aplicació (ex: "Crear endpoint de registre", "Dissenyar base de dades").
4. Assignar de manera correcta les tasques a diferents membres de l'equip de proves i simular manualment el pas d'una targeta d'un estat a un altre per a familiaritzar-se amb la interfície d'usuari i la traçabilitat del tauler.

TEMA 12: Interconnexió de Sistemes i Automatització de Notificacions



- 1 El Mecanisme dels *Webhooks* com a Motor d'Interconnexió
 - 2 Recepció Automàtica de Canvis d'Estat i Contingut
 - 3 Automatització d'Alertes i Regles de Notificació
 - 4 Fluxos de Treball Digitals i Seguretat en els Webhooks
- 🔧 Pràctica de Laboratori Proposada per a aquest tema

1 El Mecanisme dels *Webhooks* com a Motor d'Interconnexió

Perquè les plataformes de comunicació (Slack, Teams) i les eines de gestió de projectes (Jira, GitHub Projects) reben informació en temps real sobre el que passa als repositoris de codi, s'utilitza una tecnologia clau: els **Webhooks** (o ganxos web).

Definició de Webhook: És un mecanisme de comunicació automatitzat basat en esdeveniments (*event-driven*). Consisteix en una petició HTTP de tipus POST que un sistema origen (el repositori) envia cap a una URL específica d'un sistema destí (el xat o eina de gestió) de manera immediata quan ocorre un succés concret.

Diferència clau amb el Polling (Sondeig):

- **Polling (Infeicient):** El sistema destí pregunta periòdicament (cada 5 minuts) a l'origen: *"Hi ha canvis? Hi ha canvis?"*. Això consumeix recursos i genera latència.
- **Webhook (Eficient):** El sistema destí es queda escoltant de manera passiva. En el moment exacte en què es fa un *push*, el repositori avisa activament: *"Escolta, s'ha fet aquest canvi"*.

2 Recepció Automàtica de Canvis d'Estat i Contingut

Quan es connecta un repositori de codi font amb una eina de gestió de projectes, es configuren regles d'escolta automàtica de canvis. Els esdeveniments més comuns que disparen aquestes interconnexions són:

- **Creació d'una branca:** Mou de manera automàtica la tasca vinculada a la columna *"En Procés"*.
- **Obertura d'una Pull Request:** Transllada la targeta del tauler Kanban a la columna *"Revisió de Codi"* de forma desatesa.
- **Fets de Push de codi:** Sincronitza de manera immediata el contingut actualitzat, mostrant quins fitxers han canviat directament en la interfície de la tasca.

3 Automatització d'Alertes i Regles de Notificació

Dins de les directives de comunicació de l'equip de desenvolupament, és vital evitar la *"fatiga de notificacions"* (rebre milers de missatges inútils que l'equip acaba ignorant). Per això, les plataformes de comunicació es configuren aplicant regles estrictes sobre quines mètriques o alertes s'han de notificar i en quins canals:

- **Canal de Builds (#ci-pipeline-status):** Rebrà notificacions **únicament si un pipeline de CI falla** o si es recupera d'una fallada anterior. No es notifiquen les builds correctes per a evitar soroll.
- **Canal de Seguretat i Mètriques (#devsecops-alerts):** Alerta de forma immediata si l'escaneig automàtic de codi (SAST/SCA) troba una vulnerabilitat crítica o si el nivell de cobertura de test cau per davall del llindar de la *Quality Gate*.

- **Canal de Peticions de Canvi (#code-reviews):** Notifica als membres de l'equip quan es crea una nova petició de canvi (*Pull Request*) o quan hi ha una sol·licitud manual de revisió pendents.

4 Fluxos de Treball Digitals i Seguretat en els Webhooks

Atès que els Webhooks viatgen a través del protocol HTTP/HTTPS per Internet, les directives de seguretat digital exigeixen protegir aquests punts de connexió:

1. **Xifratge TLS (HTTPS):** Totes les URLs de destinació dels Webhooks han d'utilitzar obligatòriament el protocol HTTPS per a garantir que la informació viatge xifrada.
2. **Secrets de Webhook (*Webhook Secrets / Signatures*):** El repositori d'origen firma el cos de la petició HTTP (*payload*) utilitzant una clau secreta compartida amb un algorisme hash (com HMAC-SHA256). El sistema de destinació rep el missatge, calcula el hash amb el seu secret local i, si coincideix, valida que la notificació prové realment del repositori oficial i no d'un atacant que intenta suplantar-lo.



Pràctica de Laboratori Proposada per a aquest tema:

1. Crear un canal de proves en una plataforma de comunicació com **Slack** o **Discord**.
2. Anar a la configuració del canal i generar una URL de **Incoming Webhook** (Webhook d'Entrada).
3. Entrar a la configuració del teu repositori remot de GitHub o GitLab, anar a l'apartat de *Webhooks* i afegir la URL generada.
4. Seleccionar com a esdeveniment activador només el *Push* de codi i configurar un secret de validació.
5. Fer un git push des de la teua terminal local cap al repositori remot i verificar que apareix un missatge de text automàtic i instantani en el teu canal de xat informant dels canvis realitzats.

TEMA 13: Traçabilitat d'Errors i Documentació Col·laborativa



- 1 El Principi de Traçabilitat d'Errors (*Bug Tracking Traceability*)
 - 2 Relació Automatitzada de *Bugs* amb Modificacions de Codi Font
 - 3 Documentació com a Codi (*Documentation as Code – Docs-as-Code*)
 - 4 Integració de Plataformes de Documentació Col·laborativa
- 🔧 Pràctica de Laboratori Proposada per a aquest tema

1 El Principi de Traçabilitat d'Errors (*Bug Tracking Traceability*)

En entorns de producció i integració contínua, un dels majors reptes és la **resolució d'errors (*bugs*)**. Quan un usuari o un sistema de monitoratge reporta una fallada, l'equip d'enginyeria necessita saber immediatament quina línia exacta de codi ha provocat el problema, en quin moment es va introduir i quina modificació de programari la va causar. La traçabilitat bidireccional automatitzada és la capacitat de connectar directament un error registrat en l'eina de gestió de projectes amb els fitxers de codi font i els llançaments d'aplicacions (*releases*) concrets.

2 Relació Automatitzada de *Bugs* amb Modificacions de Codi Font

Mitjançant la interconnexió del repositori de codi i l'eina de seguiment de tasques estudiada en els temes anteriors, el sistema permet relacionar errors amb codi de tres maneres automatitzades:

- **Vinculació per ID d'Incidència:** Quan un desenvolupador obri una branca o fa un commit introduint l'identificador de l'error (ex: fix: corregir desbordament de memòria (BUG-104)), les plataformes connecten ambdós elements de forma nativa. Dins del tiquet de l'error apareix un enllaç al commit, i dins del commit es pot prémer per anar al tiquet.
- **Anàlisi de Culpa (*Git Blame*):** Eines integrades que permeten examinar l'historial del fitxer afectat per l'error línia per línia, identificant quin commit concret va introduir la modificació original defectuosa, qui va ser l'autor i en quina data.
- **Automatització de Peticions de Canvi (*Pull Requests vinculades*):** Enllaçar la resolució d'un bug a una Pull Request específica. El sistema impedeix tancar el tiquet de l'error fins que el pipeline de CI no haja validat el codi de la PR i s'haja fusionat de manera correcta en la branca estable.

3 Documentació com a Codi (*Documentation as Code - Docs-as-Code*)

L'enfocament modern de la documentació tècnica en equips DevOps segueix la mateixa filosofia que la infraestructura (IaC): **la documentació es tracta exactament igual que el codi font**.

- Es redacta en llenguatges de marcatge lleugers i universals com **Markdown** (.md) o *AsciiDoc*.
- Es guarda dins del mateix repositori de codi de l'aplicació (en una carpeta com /docs).
- Està subjecta al control de versions (Git), permetent fer revisions de text, gestionar branques i auditar canvis reals d'estat.

El Pipeline de Documentació en la CI:

Quan es fa un canvi en la branca estable, el servidor d'Integració Contínua executa un pipeline específic per a la documentació: passa un linter de text (com *Markdownlint*), compila els fitxers en un lloc web estàtic utilitzant generadors (com *MkDocs*, *Docusaurus* o *Hugo*) i publica la documentació actualitzada de manera immediata i automàtica en un servidor intern o plataforma web.

4 Integració de Plataformes de Documentació Col·laborativa

A més dels fitxers Markdown en el repositori, els equips utilitzen plataformes de documentació col·laborativa com **Confluence (Atlassian)** o **Wikis integrades (GitHub/GitLab Wikis)**.

Sinergies d'interconnexió:

- **Mètriques de CI a la Wiki:** Es configuren paràmetres perquè el servidor de CI publiqui de manera automàtica taules de cobertura de test, informes de dependències o estadístiques de qualitat directament en les pàgines de documentació de l'equip.
- **Estat de Tasques en Temps Reial:** Permet incrustar llistats de bugs actius, gràfics de velocitat de resolució (*burndown charts*) i l'estat dels últims desplegaments directament dins de la documentació del projecte, assegurant que la informació es mantinga constantment actualitzada sense intervenció manual.

Pràctica de Laboratori Proposada per a aquest tema:

1. Crear una carpeta anomenada docs/ en el teu repositori i afegir un fitxer Markdown (ex: arquitectura.md) amb documentació tècnica bàsica de l'aplicació.
2. Simular un flux de treball de *Docs-as-Code*: fer una modificació en la documentació local des del teu editor de text.
3. Fer un commit i una Pull Request amb els canvis de text de forma idèntica a com es faria amb un fitxer de codi.
4. Verificar des de la interfície de la plataforma la seua correcta visualització en l'apartat de codi font i veure la traçabilitat de l'historial de versions del document tècnic.